Progetto di Scalable and Cloud Programming A.A. 2025-26

Leonardo Venturi (matricola 0001184072)

Codice: `https://github.com/Leo3812/SCP_Project`

1 Introduzione

Il progetto analizza un dataset di eventi sismici con l'obiettivo di determinare la coppia di coordinate con il massimo numero di giorni di co-occorrenza, e di elencare le date in ordine cronologico. Le coordinate geografiche sono ottenute arrotondando latitudine e longitudine a una singola cifra decimale; la finestra temporale è definita su base giornaliera.

Sono stati implementati tre approcci differenti in Scala con Spark (basati sul paradigma MapReduce). Gli esperimenti sono poi stati eseguiti su Google Cloud Dataproc con 2, 3 e 4 worker, allo scopo di confrontare le prestazioni e valutarne la scalabilità.

2 Implementazione

2.1 Pre-processing

Il file CSV viene letto con uno schema esplicito (`longitude`, `latitude`: `Double`; `date`: `String`). Ogni riga del dataset diventa quindi una coppia `(data, Coord(lon, lat))`: le coordinate sono arrotondate a un decimale mediante `round(x*10)/10.0`, e la data viene normalizzata troncandola ai primi 10 caratteri della stringa.

L'RDD risultante è memorizzato in cache con livello (`MEMORY_AND_DISK`) e materializzato con `count()` prima di avviare il timer. In questo modo la lettura e il parsing non rientrano nelle misure. La deduplicazione delle coppie `(data, cella)` causate dall'arrotondamento, viene effettuata all'interno di ciascun approccio.

2.2 Approccio 1 (Baseline)

1. Si applica `distinct()` su `(data, cella)` per rimuovere i duplicati introducendo il primo shuffle.
2. Le celle vengono raggruppate per date tramite `groupByKey` (secondo shuffle). Questo è seguito dall'applicazione di un filtro che scarta le date con una sola cella, poiché non possono generare coppie.
3. Per ogni data si generano tutte le coppie di celle distinte, imponendo un ordinamento lessicografico.
4. Si raggruppano utilizzando `groupByKey` per coppia (terzo shuffle).
5. Si seleziona infine la coppia associata al maggior numero di date mediante una `reduce`.

Questo approccio fa quindi uso di tre shuffle. Nel quarto passo ogni record contiene una data e ogni coppia accumula in memoria la lista completa delle sue date. Questo comporta un costo di memoria e di elaborazione superiore a quello necessario per individuare la coppia più frequente, operazione per la quale sarebbe sufficiente contarne le occorrenze.

2.3 Approccio 2: conteggio e recupero delle date

1. Si usa `aggregateByKey()` al fine di aggregare per data le coordinate in un `HashSet`. In questo modo deduplicazione e raggruppamento avvengono nello stesso shuffle. Le date associate a una sola cella sono scartate come per il precedente approccio, e il risultato viene tenuto in cache.
2. Si producono quindi le coppie di celle per data producendo record del tipo `((a,b), 1)`. Utilizzando gli iteratori si evita di costruire una collezione intermedia per ogni data.
3. Si applica ora `combineByKey` con `HashPartitioner()` per sommare le occorrenze di ciascuna coppia (secondo shuffle). Il map-combiner è disabilitato (`mapSideCombine = false`) , poiché nelle diverse prove effettuate ha prodotto tempi maggiori. Probabilmente ciò è dovuto al fatto che le stesse coppie si ripetano poco all'interno di una partizione e il combiner aggiunga solo costo di hashing e memoria.
4. Si individua quindi la coppia con il conteggio massimo mediante `reduce`.
5. Si effettua una seconda scansione dell'RDD in cache: si filtrano le date il cui insieme di celle contiene entrambe le celle vincenti, si raccolgono sul driver e si ordinano.

L'approccio utilizza due shuffle. Il numero dei record generati durante la costruzione delle coppie è lo stesso dell'Approccio 1, ma i valori trasferiti sono interi anziché date. Inoltre, dopo lo shuffle si aggregano interi, evitando di costruire e mantenere in memoria liste complete di date per ogni coppia. Le date vengono recuperate soltanto per la coppia vincente.

2.4 Approccio 3: pruning con lower bound

Questo approccio sperimentale mira a ridurre lo spazio di ricerca prima di eseguire la pipeline di conteggio delle coppie.

1. Si utilizza `reduceByKey` per contare il numero di record in cui compare ogni coordinata.
2. Si individuano le celle con il maggior numero di occorrenze mediante `takeOrdered`. Il lower bound L è definito come il numero di date in cui compaiono entrambe le celle. Si filtrano dunque i record delle due celle, si raggruppano per data e si contano le date in cui sono presenti entrambe
3. Si raccolgono sul driver le celle con conteggio $\geq L$ e si filtra l'RDD di input tenendo solo i record di queste celle (`persist`).
4. Sui dati ridotti si esegue la pipeline dell'Approccio 1 che richiede 3 shuffle.

Correttezza del pruning. La coppia individuata al secondo passaggio fornisce una soluzione ammissibile con L giorni di co-occorrenza. Di conseguenza, la coppia ottima deve avere un conteggio almeno pari a L . Se una cella appartiene alla coppia ottima, deve quindi comparire in almeno L giorni distinti. Poiché il conteggio delle occorrenze della cella è almeno pari al numero di giorni in cui essa compare, entrambe le celle della coppia ottima superano la soglia del pruning. Nessuna delle due viene pertanto eliminata e la soluzione ottima rimane nel dataset filtrato.

3 Setup sperimentale

3.1 Piattaforma.

I diversi approcci sono stati testati su cluster Dataproc con macchine `n2-standard-4` (4 vCPU, 16 GB) per master e worker, dischi da 240 GB, regione europe-west3, Spark 3.5.3 e Scala 2.12.18. Il dataset completo è stato caricato su un bucket Cloud Storage, mentre il cluster viene gestito con uno script Python che automatizza il testing.

3.2 Esperimenti.

- **Sensibilità alle partizioni**: i tre approcci sono stati testati su cluster con 4 worker e 16, 32, 64, 128 e 256 partizioni.
- **Strong scaling**: testato con 2, 3 e 4 worker, con 128 partizioni per il primo approccio, 64 per il secondo e 16 per il terzo.

Metodo di misura.
Ogni configurazione è stata eseguita 3 volte. Riportiamo la mediana con la deviazione standard indicata da barre (interpretate con cautela per il numero di ripetizioni effettuate). Il timer viene avviato dopo la lettura del dataset, la memorizzazione in cache e la materializzazione dell'RDD mediante count(). I tempi di lettura del CSV e di inizializzazione di Spark sono quindi esclusi. Per l'Approccio 3, tutte le operazioni di pruning sono invece incluse nel tempo misurato.

4 Risultati

4.1 Risultato dell'analisi

La coppia di coordinate con più co-occorrenze si è rivelata essere $((38.8, -122.8), (38.8, -122.7))$, presente in 10032 giorni distinti.

4.2 Sensibilità alle partizioni

La Figura 1 e la Tabella 1 riportano i tempi di esecuzione dei tre approcci su quattro worker, al variare del numero di partizioni. Possiamo osservare come il primo approccio sia il più sensibile alla variazione delle partizioni.Inoltre sia il primo che il secondo approccio subiscono un decremento delle prestazioni all'aumentare del numero di partizioni oltre le 128, l'overhead di pianificazione e gestione dei task probabilmente ha superato i benefici dati dalla parallelizzazione.

L'Approccio 3 raggiunge le prestazioni migliori con 16 partizioni. In questo caso, il pruning riduce notevolmente il volume di dati da elaborare, rendendo meno conveniente suddividere il lavoro in un numero elevato di task.

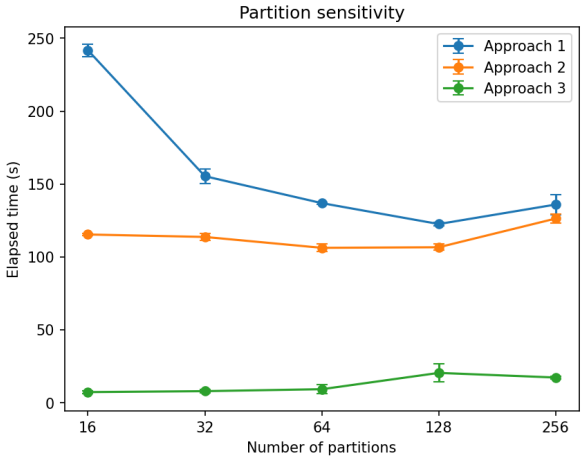


Figura 1: Tempo di esecuzione al variare delle partizioni (4 worker, mediana di 3 prove).

Tabella 1: Partizioni migliori e peggiori su 4 worker (mediana).

Approccio	Migliore	Tempo (s)	Peggiore	Tempo (s)
1	128	122.46	16	241.77
2	64	106.14	256	124.14
3	16	7.08	128	20.27

4.3 Strong scaling.

Lo strong scaling misura come varia il tempo di esecuzione al crescere delle risorse computazionali, mantenendo fisso il problema da risolvere. Nel nostro caso, lo speedup e l'efficienza sono calcolati rispetto alla configurazione con due worker, conseguentemente:

$$S(n) = \frac{T(2)}{T(n)} \qquad E(n) = \frac{S(n)}{(n/2)}$$

La Tabella 2 riporta i tempi assoluti e la Tabella 3 speedup ed efficienza (Figura 2).

Tabella 2: Tempi assoluti, mediana [min–max] in secondi.

Approccio	Partizioni	2 worker	3 worker	4 worker
1	128	173.31 [162.07–190.63]	165.25 [154.62–165.26]	131.60 [130.42–132.41]
2	64	145.39 [139.96–148.84]	109.22 [107.75–109.41]	105.55 [105.48–107.06]
3	16	11.81 [8.88–12.52]	12.20 [9.75–15.10]	11.81 [9.18–15.82]

Tabella 3: Speedup ed efficienza rispetto a 2 worker.

Approccio	$S(3)$	$S(4)$	$E(3)$	$E(4)$
1	1,05	1,32	0,70	0,66
2	1,33	1,38	0,89	0,69
3	0,97	1,00	0,65	0,50

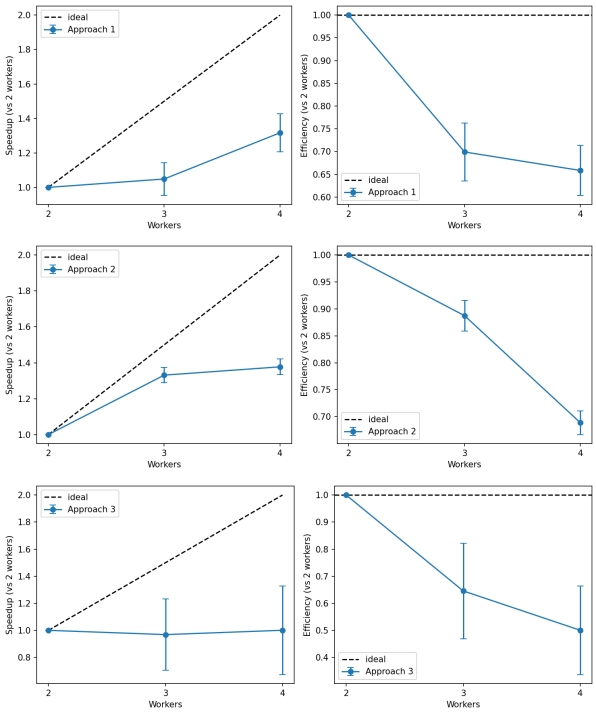


Figura 2: Speedup (sinistra) ed efficienza (destra) rispetto a 2 worker: Approcci 1, 2 e 3.

L'Approccio 1 migliora all'aumentare del numero di worker, ma lo speedup rimane contenuto. L'Approccio 2 presenta un miglioramento più marcato nel passaggio da due a tre worker, mentre il beneficio aggiuntivo ottenuto con quattro worker è limitato. L'Approccio 3 non mostra invece uno speedup apprezzabile: i tempi, già ridotti grazie al pruning risultano estremamente variabili per la brevità del calcolo.

Questi risultati possono essere anche interpretati mediante la legge di Amdahl, secondo cui l'incremento delle risorse di calcolo non produce necessariamente un miglioramento proporzionale delle prestazioni. La presenza di parti non parallelizzabili, insieme ai costi di gestione delle operazioni distribuite, può limitare lo speedup ottenibile. Questo effetto è particolarmente rilevante quando il tempo di elaborazione è ridotto, come nel terzo approccio, perché i costi fissi e la variabilità delle misurazioni possono incidere maggiormente sul tempo totale.

4.4 Limiti

Le misure si basano su tre esecuzioni per configurazione; le differenze contenute, soprattutto per l'Approccio 3, non possono quindi essere considerate conclusive.

5 Conclusione

I tre approcci producono lo stesso risultato, ma presentano costi computazionali molto diversi. L'Approccio 3, grazie al pruning basato su un lower bound, è il più rapido: la riduzione dello spazio di ricerca limita il lavoro necessario per generare e aggregare le coppie. Il vantaggio è tale che i costi fissi diventano predominanti e l'aumento del numero di worker non produce miglioramenti apprezzabili.

Tra gli approcci che elaborano il dataset senza pruning, l'Approccio 2 è preferibile alla baseline: evita la costruzione di liste complete di date, aggrega direttamente i conteggi e mostra una minore sensibilità al numero di partizioni. In entrambi i casi, tuttavia, lo speedup rimane limitato, attestandosi intorno a 1,3–1,4× con quattro worker.

Nel complesso, i risultati evidenziano come l'ottimizzazione algoritmica possa avere un impatto molto maggiore del semplice aumento delle risorse computazionali. La scelta dell'approccio deve quindi tenere conto non soltanto del tempo di esecuzione, ma anche del costo delle operazioni distribuite, della memoria richiesta e del carico residuo disponibile per il parallelismo.